Producer-Consumer

ducer-Consumer · Bounded queue between encoder and uploader · Build with Wa

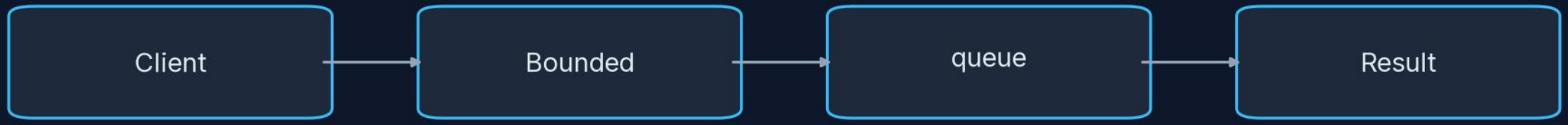
Today's Hook

- Problem: Bounded queue between encoder and uploader
- Pattern: Producer-Consumer — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Bounded queue between encoder and uploader
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Producer-Consumer: Bounded queue between encoder and uploader

Real-World Scenario

- Running example: Bounded queue between encoder and uploader
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```java
package com.buildwithwaqar.patterns.concurrency.producerconsumer;

import com.buildwithwaqar.patterns.concurrency.producerconsumer.BoundedQueue;
import java.util.concurrent.*;
/** Problem: encoder produces frames; uploader consumes without unbounded memory. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Producer-Consumer problem: bounded frame queue ===");
    var q = new BoundedQueue<Integer>(2);
    try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
      pool.submit(() -> { for (int i=1;i<=4;i++){ q.put(i); System.out.println("encode "+i);} return null; });
      var sum = pool.submit(() -> { int s=0; for(int i=0;i<4;i++){ int v=q.take(); System.out.println("upload "+v); s+=v;} return s;});
      System.out.println("checksum=" + sum.get());
    }
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.concurrency.producerconsumer;

import java.util.ArrayDeque;
import java.util.Queue;

public final class BoundedQueue<T> {
    private final Queue<T> q = new ArrayDeque<>();
    private final int capacity;

    public BoundedQueue(int capacity) { this.capacity = capacity; }

    public synchronized void put(T item) throws InterruptedException {
        while (q.size() == capacity) wait();
        q.add(item);
        notifyAll();
    }

    public synchronized T take() throws InterruptedException {
        while (q.isEmpty()) wait();
        var item = q.remove();
        notifyAll();
        return item;
    }
```

Solution (2)

```
    public synchronized int size() { return q.size(); }
}
```

Runnable main

```java
package com.buildwithwaqar.patterns.concurrency.producerconsumer;

import com.buildwithwaqar.patterns.concurrency.producerconsumer.BoundedQueue;
import java.util.concurrent.*;
/** Problem: encoder produces frames; uploader consumes without unbounded memory. */
public final class UseCaseDemo {
  public static void main(String[] a) throws Exception {
    System.out.println("=== Producer-Consumer problem: bounded frame queue ===");
    var q = new BoundedQueue<Integer>(2);
    try (var pool = Executors.newVirtualThreadPerTaskExecutor()) {
      pool.submit(() -> { for (int i=1;i<=4;i++){ q.put(i); System.out.println("encode "+i);} return null; });
      var sum = pool.submit(() -> { int s=0; for(int i=0;i<4;i++){ int v=q.take(); System.out.println("upload "+v); s+=v;} return s;});
      System.out.println("checksum=" + sum.get());
    }
  }
}
```

Walkthrough Output

```
=== Producer-Consumer problem: bounded frame queue ===
encode 1
encode 2
encode 3
upload 1
upload 2
upload 3
encode 4
upload 4
checksum=10
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Producer-Consumer.

Related Patterns

- Contrast Producer-Consumer with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Producer-Consumer → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.